

What we're going to cover:

- Functions
- *arg/**kwargs
- NameSpace local/global scope
- generators

توابع پایتون

Functions

تابع، بلوکی از کد است که وظیفه‌ی خاصی را انجام می‌دهد

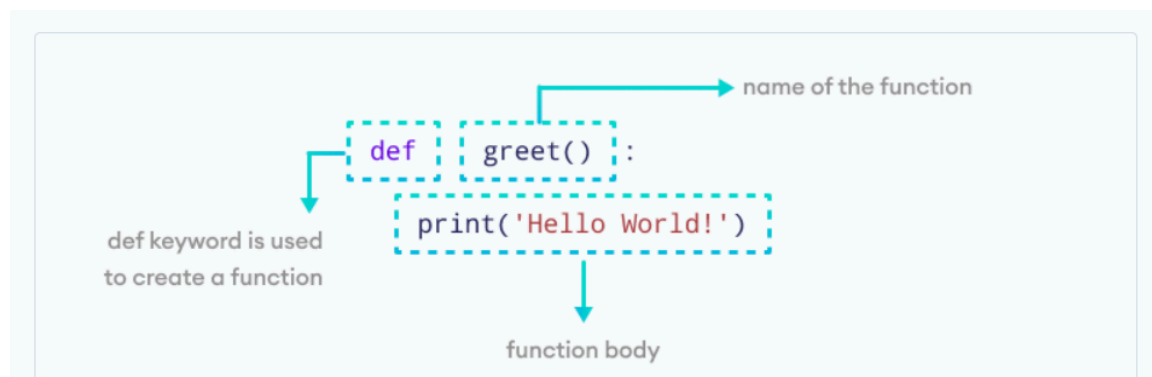
اگر پیش‌بینی می‌کنید که لازم است کدی یکسان یا بسیار مشابه را بیش از یک بار تکرار کنید

شاید بهتر باشد یک تابع قابل‌استفاده مجدد بنویسید

توابع همچنین می‌توانند با اختصاص نامی به مجموعه‌ای از دستورات پایتون، به خواناتر شدن کد شما کمک کنند

ایجاد یک تابع

```
In [1]: def greet():  
        print('Hello World!')
```



فراخوانی یک تابع

در مثال بالا، ما تابعی به نام `greet()` تعریف کرده‌ایم

اگر کد بالا را اجرا کنیم، خروجی دریافت نخواهیم کرد

به این دلیل که ایجاد یک تابع به این معنی نیست که ما کد داخل آن را اجرا می‌کنیم

به این معنی است که کد برای استفاده ما در صورت تمایل وجود دارد.

برای استفاده از این تابع، باید تابع را فراخوانی کنیم

```
In [1]: def greet():
        print('Hello World!')

        # call the function
        greet()

        print('Outside function')
```

Hello World!
Outside function



```
In [1]: # can type some comments inside function
        # type the function names in snake_case
        def hello():
            """
            :return:
            """
            print("Hello World!")
        hello()
```

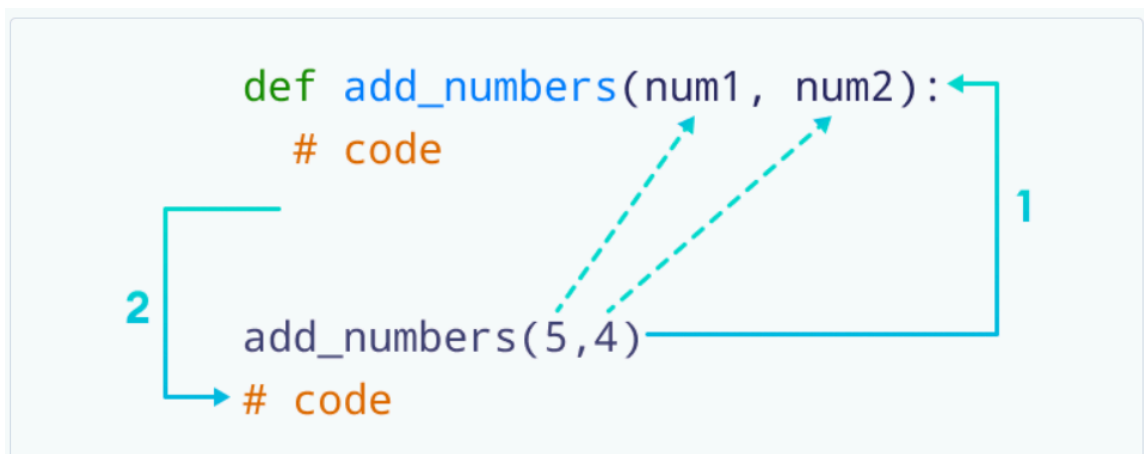
Hello World!

تابع جمع دو عدد

```
In [4]: # function with two arguments
        def add_numbers(num1, num2):
            sum = num1 + num2
            print("Sum: ", sum)

        # function call with two values
        add_numbers(5, 4)
```

Sum: 9



دستور return

ما با استفاده از دستور return مقداری را از تابع برمی‌گردانیم

```
In [5]: # function definition  
def find_square(num):  
    result = num * num  
    return result  
  
# function call  
square = find_square(3)  
  
print('Square:', square)
```

Square: 9

در مثال بالا، تابعی به نام `find_square()` ایجاد کرده‌ایم

این تابع یک عدد را دریافت کرده و مجذور آن را برمی‌گرداند

همچنین نشان‌دهنده پایان یافتن تابع است. هیچ کدی پس از `return` اجرا نمی‌شود
نکته: دستور `return`

```
In [1]: # Exercise:  
# count characters  
def count_char(n, c):  
    """  
    :param n: is the name  
    :param c: is the character  
    :return: the number of character  
    """  
    count = 0  
    for char in n:  
        if char == c:  
            count += 1  
    return count
```

```
name = 'peter griffen'
result = count_char(name, 'e')
print(result)
```

3

Return Statement	Print Statement
It is used to exit a function and return a value	It is used to display output to the console
It returns a value that can be assigned to a variable or used in any expression	It displays output to the console but does not return the value
It can be used multiple times in a function but only one value can be returned at a time	It can be used multiple times in a function but does not affect the function's return value
Exiting the function with return ends the function and control returns to calling code	The print does not affect program flow and execution continues normally
Example: return sum in a function that calculates the sum of two numbers	Example: print("Hello, world!") to display a message on the console

Reference: [GeeksforGeeks](#)

Keyword Argument (named arguments)

- آرگومان‌های کلیدواژه‌ای بدین معناست که هنگام فراخوانی تابع، مقادیر را با استفاده از نام پارامترها ارسال می‌کنید
- تا زمانی که نام‌ها صحیح باشند، ترتیب اهمیتی ندارد
- اگر برخی از آرگومان‌ها ارائه نشوند، از مقادیر پیش‌فرض استفاده می‌شود
- این کار باعث می‌شود کد خواناتر و کمتر مستعد خطا باشد

```
In [8]: def nameAge(name, age):
        print("Hi, I am", name)
        print("My age is ", age)

        nameAge(name="Prince", age=20)
        print('---'*10)
        nameAge(age=20, name="Prince")
```

Hi, I am Prince

My age is 20

Hi, I am Prince

My age is 20

Positional Arguments (آرگومان‌های موقعیتی)

- آرگومان‌های موقعیتی بدین معناست که مقادیر دقیقاً به همان ترتیبی که پارامترها در تابع تعریف شده‌اند، به آن ارسال می‌شوند
- اولین مقدار به اولین پارامتر، دومین مقدار به دومین پارامتر و به همین ترتیب اختصاص می‌یابد
- تغییر ترتیب می‌تواند به نتایج غیرمنتظره‌ای منجر شود

```
In [1]: def add(a, b):
        return a + b

        # 5 goes to a, 10 goes to b
        print(add(5, 10))
```

15

Arbitrary arguments (variable-length arguments *args and **kwargs)

آرگومان‌های دلخواه

در پایتون، آرگومان‌های دلخواه به ما امکان می‌دهند تعداد متغیری از آرگومان‌ها را به یک تابع ارسال کنیم

این قابلیت زمانی مفید است که از پیش ندانیم به ارسال چه تعداد آرگومان نیاز خواهیم داشت. دو نوع آرگومان دلخواه وجود دارد:

1- *args in Python (Non-Keyword Arguments):

- آرگومان‌های موقعیتی اضافی ارسال شده به یک تابع را در یک تاپل جمع‌آوری می‌کند

2- **kwargs in Python (Keyword Arguments):

- آرگومان‌های کلیدواژه‌ای اضافی ارسال شده به یک تابع را در یک دیکشنری جمع‌آوری می‌کند

```
In [6]: def multiply(*args):
        result = 1
        for i in args:
            result *= i
        return result

        res = multiply(2, 3, 4)
        print(res)
```

24

```
In [7]: def myfunc(*args):
        print("Non-Keyword Arguments (*args):")
```

```
print(args)

myfunc('hi', 'peter')
```

Non-Keyword Arguments (*args):
(*'hi'*, *'peter'*)

```
In [4]: # Python program to illustrate
# *args for variable number of arguments
def myFun(*args):
    for arg in args:
        print(arg)

# Driver code with different arguments
myFun('Python', 'is', 'amazing')
```

Python
is
amazing

```
In [10]: def area(**kwargs):
          print(kwargs)

          area(3, 4)
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[10], line 4
      1 def area(**kwargs):
      2     print(kwargs)
      3
----> 4 area(3, 4)

TypeError: area() takes 0 positional arguments but 2 were given
```

```
In [11]: # The above code is wrong because we have to mention the positions
def area(**kwargs):
    print(kwargs)

    area(a = 3, b = 4)

{'a': 3, 'b': 4}
```

```
In [5]: # Python program to illustrate
# **kwargs for variable number of keyword arguments
def fun(**kwargs):
    for key, value in kwargs.items():
        print(f"{key}: {value}")

# Driver code
fun(course="DSA", platform="GeeksforGeeks", difficulty="easy")
```

course: DSA
platform: GeeksforGeeks
difficulty: easy

```
In [12]: def calculate(**kwargs):
          print(f'kwargs are {kwargs}')
          if 'width' in kwargs:
              return kwargs['width'] * kwargs['height']
          if 'radius' in kwargs:
              return kwargs['radius'] * kwargs['radius'] * 3.1415

          result = calculate(width = 4, height = 5)
          print(f'the area is = {result}')
          circle = calculate(radius = 3)
          print(f'the circle area is = {circle}') # add :1.2f to circle
```

```
kwargs are {'width': 4, 'height': 5}
the area is = 20
kwargs are {'radius': 3}
the circle area is = 28.273500000000002
```

Global Namespace:

فضای نام سراسری

شامل نام‌هایی است که در سطح بالای برنامه، خارج از تمامی توابع و کلاس‌ها، تعریف شده‌اند. معمولاً می‌توان از هر نقطه‌ای در همان مازول به این نام‌ها دسترسی داشت

تابع `show_score()` می‌تواند به `score` دسترسی داشته باشد

زیرا نام‌های سراسری در داخل توابع قابل مشاهده هستند، مگر اینکه متغیر محلی‌ای با همان نام وجود داشته باشد

See the below example:

```
In [19]: score = 95
          def show_score():
              print(score)

          show_score()
```

95

Local Namespace:

فضای نام محلی

هر زمان که تابعی فراخوانی می‌شود، ایجاد می‌گردد. متغیرهای تعریف شده در داخل تابع در این فضای نام ذخیره می‌شوند. این متغیرها تنها در حین اجرای تابع در دسترس هستند

age به فضای نام محلی `show_age()` تعلق دارد

تنها از داخل آن تابع می‌توان به آن دسترسی داشت

پس از پایان اجرای تابع، فضای نام محلی حذف می‌شود

See below example:

```
In [20]: def show_age():  
         age = 25  
         print(age)  
  
show_age()
```

25

Built-in Namespace:

شامل نام‌هایی است که به طور خودکار در هر برنامه پایتون در دسترس هستند
این نام‌ها توسط خود پایتون فراهم شده‌اند، بنابراین می‌توانید بدون تعریف کردنشان از آن‌ها استفاده کنید

print() یک تابع داخلی است که برای نمایش خروجی استفاده می‌شود

len() یک تابع داخلی است که برای یافتن طول یک شیء استفاده می‌شود

از آنجا که این توابع به فضای نام داخلی تعلق دارند، می‌توان آن‌ها را مستقیماً در هر برنامه پایتون به کار برد

See the below example:

```
In [21]: print("Hello")  
         print(len("Python"))
```

Hello

6

Namespaces

فضای نام مجموعه‌ای از نام‌های نمادین تعریف شده در لحظه، به همراه اطلاعاتی درباره شیئی است که هر نام به آن ارجاع می‌دهد

می‌توانید فضای نام را همچون یک دیکشنری در نظر بگیرید که در آن کلیدها همان نام‌های اشیاء و مقادیر، خود آن اشیاء هستند

در یک برنامه پایتون، چهار نوع فضای نام وجود دارد:

- Built-In
- Global
- Enclosing

- Local

The LEGB Rule for Searching Names

با بازگشت به پرسش بخش قبل، فرض کنید که در حال ارجاع به x از درون یک تابع داخلی هستید

در این حالت، پایتون به ترتیب زیر به دنبال x می‌گردد

Local: Python searches for x inside the inner function. If it doesn't find x there, then it moves to the enclosing scope.

Enclosing: Next, Python searches for x in the enclosing function's scope. If it's not found there, then it moves to the global scope.

Global: Python searches for x in the global scope. If it still doesn't find it, it moves to the built-in scope.

Built-in: Finally, Python searches for x in the built-in scope. If it doesn't find x there, then it raises a `NameError` exception.

```
In [13]: x = "global"

def outer():
    def inner():
        print(x)
    inner()

outer()
```

global

```
In [14]: x = "global"

def outer():
    x = "enclosing"
    def inner():
        print(x)
    inner()

outer()
```

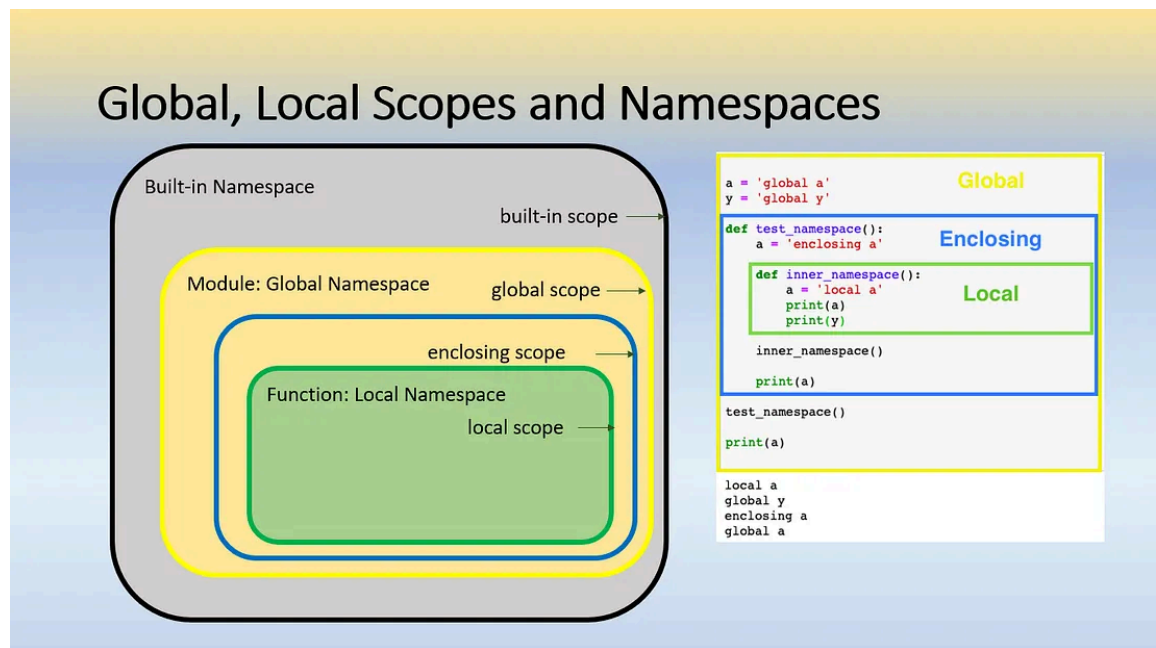
enclosing

```
In [15]: x = "global"

def outer():
    x = "enclosing"
    def inner():
        x = "local"
        print(x)
    inner()
```

outer()

local



Reference:Medium

Generators

بسیاری از اشیاء در پایتون از قابلیت پیمایش پشتیبانی می‌کنند؛ برای مثال، پیمایش روی عناصر یک لیست یا خطوط یک فایل

این کار به واسطه‌ی پروتکل تکرارگر انجام می‌شود که روشی کلی برای قابل پیمایش ساختن اشیاء است

A generator

روشی مناسب — مشابه نوشتن یک تابع معمولی — برای ایجاد یک شیء قابل تکرار جدید است

برخلاف توابع معمولی که اجرا می‌شوند و در هر بار فراخوانی تنها یک نتیجه برمی‌گردانند

ژنراتورها می‌توانند با متوقف کردن و سپس ازسرگیری اجرا در هر بار استفاده، دنباله‌ای از چندین مقدار را بازگردانند

برای ایجاد یک ژنراتور، در بدنه تابع به جای کلمه کلیدی `return` از `yield` استفاده کنید

```
In [24]: # When you actually call the generator, no code is immediately executed:
def squares(n=10):
    print(f"Generating squares from 1 to {n ** 2}")
    for i in range(1, n + 1):
        yield i ** 2
```

```
gen = squares()
print(gen)
```

<generator object squares at 0x000002A129DC1E00>

```
In [25]: for x in gen:
          print(x, end=' ')
```

Generating squares from 1 to 100

1 4 9 16 25 36 49 64 81 100

Note:

از آنجا که ژنراتورها خروجی را عنصر به عنصر تولید می‌کنند (در مقابل تولید یک‌جای کل لیست)، این ویژگی می‌تواند به برنامه شما کمک کند تا حافظه کمتری مصرف کند

Why Do We Need Generators?

- Memory Efficient :
 - مدیریت داده‌های حجیم یا نامحدود بدون بارگذاری کامل آن‌ها در حافظه
- No List Overhead :
 - اقلام را یکی‌یکی ارائه دهید و از ایجاد فهرست کامل خودداری کنید
- Lazy Evaluation :
 - مقادیر را تنها در صورت نیاز محاسبه کنید تا عملکرد بهبود یابد
- Support Infinite Sequences :
 - برای تولید داده‌های نامحدود، مانند دنباله فیبوناچی، ایده‌آل است
- Pipeline Processing :
 - ژنراتورها را به هم متصل کنید تا داده‌ها را به صورت مرحله‌ای و کارآمد پردازش کنند

yield vs Return

Yield:

این دستور در توابع مولد برای ارائه دنباله‌ای از مقادیر در طول زمان به کار می‌رود

با اجرای دستور yield عملکرد تابع متوقف شده، مقدار فعلی بازگردانده می‌شود

این ویژگی به تابع اجازه می‌دهد تا هنگام فراخوانی مجدد، کار خود را از همان نقطه قبلی ادامه دهد؛ امری که آن را به گزینه‌ای ایده‌آل برای تولید کارآمد دنباله‌های بزرگ یا پیچیده تبدیل می‌کند

Return:

از آن برای خروج از یک تابع و بازگرداندن یک مقدار نهایی استفاده می‌شود

به محض اجرای return تابع بلافاصله خاتمه می‌یابد و هیچ وضعیتی حفظ نمی‌شود
دستور

```
In [33]: def fibonacci_numbers(nums):
          x, y = 0, 1
          for _ in range(nums):
              x, y = y, x + y
              yield x

          # we can print all in a list
          # print(list(fibonacci_numbers(10)))
          # or we can create it one by one
          fib = fibonacci_numbers(5)
          print(next(fib))
          # Print next multiple times to see the result!
```

1